

# SIGHW硬件小组第八次活动

## AXI4 burst模式读写DDR3

2026年6月6日

高嘉豪

# AXI4 与 AXI4 Lite 的区别

下面是AXI4接口多出的信号,并按功能分成了几大类

- 1. ID信号(每通道一个):用于标识传输序号,以便于支持乱序
- 2. USER信号(每通道一个):用于用户扩展
- 3. 内存类型(ARCACHE,AWCACHE):指示bufferable和cacheable
- 4. 锁(AWLOCK,ARLOCK):用于支持原子操作
- 5. 服务质量(AWQOS,ARQOS):某种程度上可以认为是优先级(协议不规定解码方式)
- 6. 区域信息(AWREGION,ARREGION):多片选逻辑的解码
- 7. 保护类型(AWPROT,ARPROT): 指示事务的特权级和安全属性等

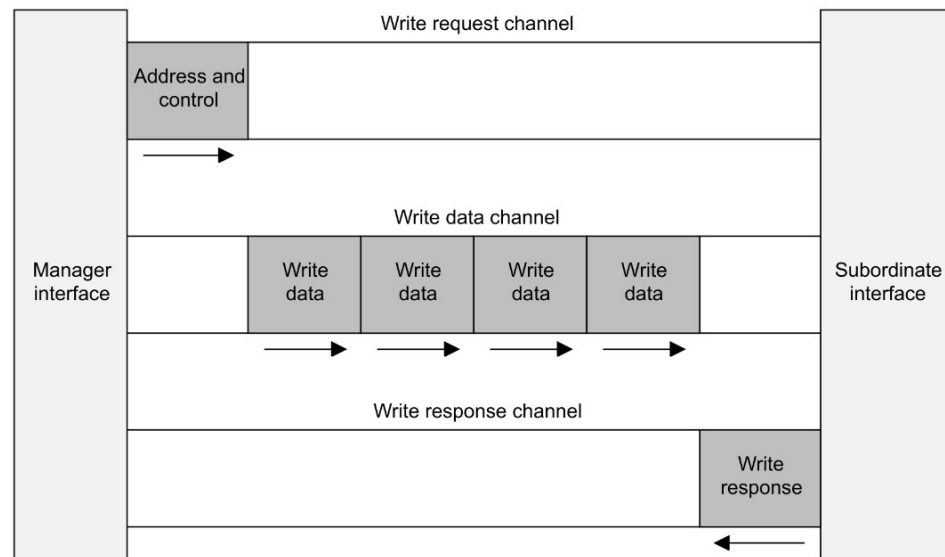
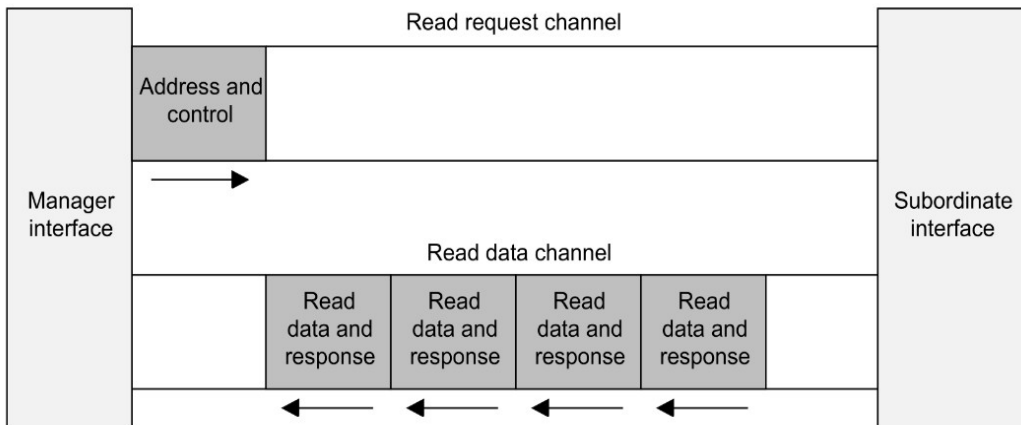
这七类特性,和Burst无关

# 突发传输

一个拍就是一个Write data(带上握手)

一个burst就是图中的一个全过程

故burst由多拍组成



# 突发传输

与burst相关的内容:

- 1. 突发长度(LEN):指定一次突发传输包含的数据节拍数(1-256)
- 2. 突发大小(SIZE):指定每个拍内传输的字节数(2的幂次,不超过128)
- 3. 突发类型(BURST):指定地址的计算方式
- 4. 最后指示器(LAST):指定当前节拍是最后一拍

扩展了的内容:

- RESP:指示传输状态(OKAY/EXOKAY/SLVERR/DECERR)

# Valid Ready 握手

- 一个时钟周期内, Valid & Ready = 1
- 在这个时钟周期内的 information 会被采样

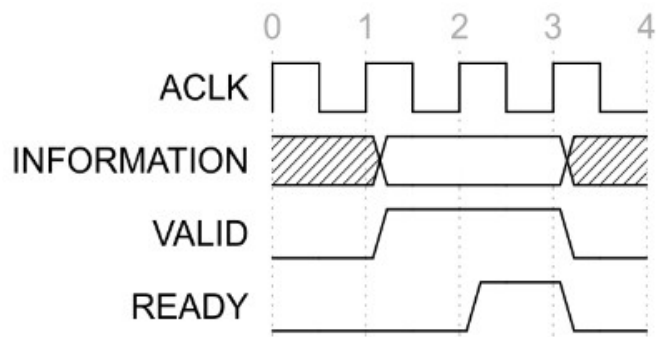


Figure A2.2: VALID before READY handshake

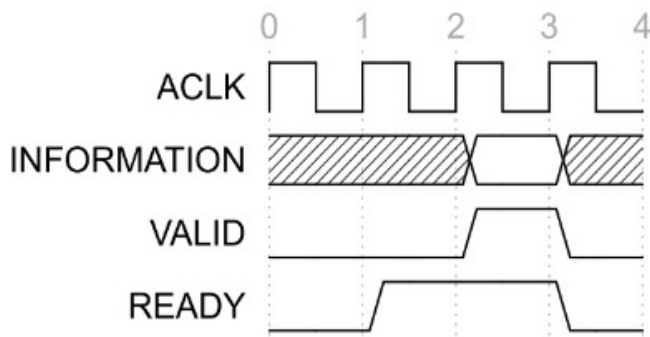


Figure A2.3: READY before VALID handshake

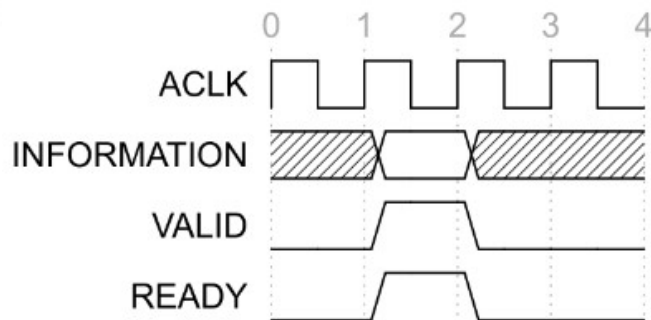
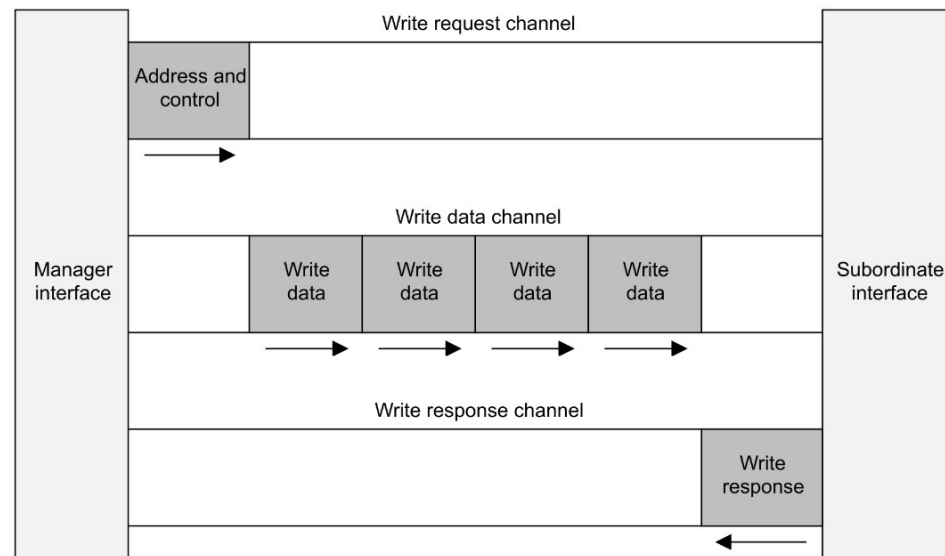


Figure A2.4: VALID with READY handshake

# 写操作的过程

- 1. AWADDR,AWLEN,AWSIZE  
拉高AWVALID握手
- 2. 循环AWSIZE次(在最后一次时拉高WLAST)  
准备WDATA,WSTRB,拉高WVALID握手
- 3. 拉高BREADY握手,检查WRRESP

这期间,只要不拉高WVALID,从机就会一直等



# 读操作的过程

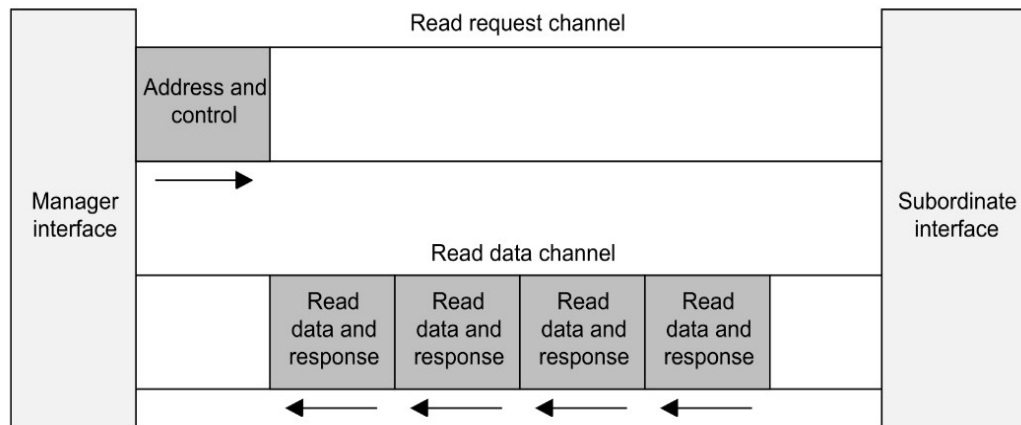
- 1. ARADDR,ARLEN,ARSIZE

拉高ARVALID握手

- 2. 循环直到一拍中RLAST被拉高

拉高RREADY,在握手的周期,采样RDATA

这期间,只要不拉高RREADY,从机就会一直等



# Burst访问地址的确定

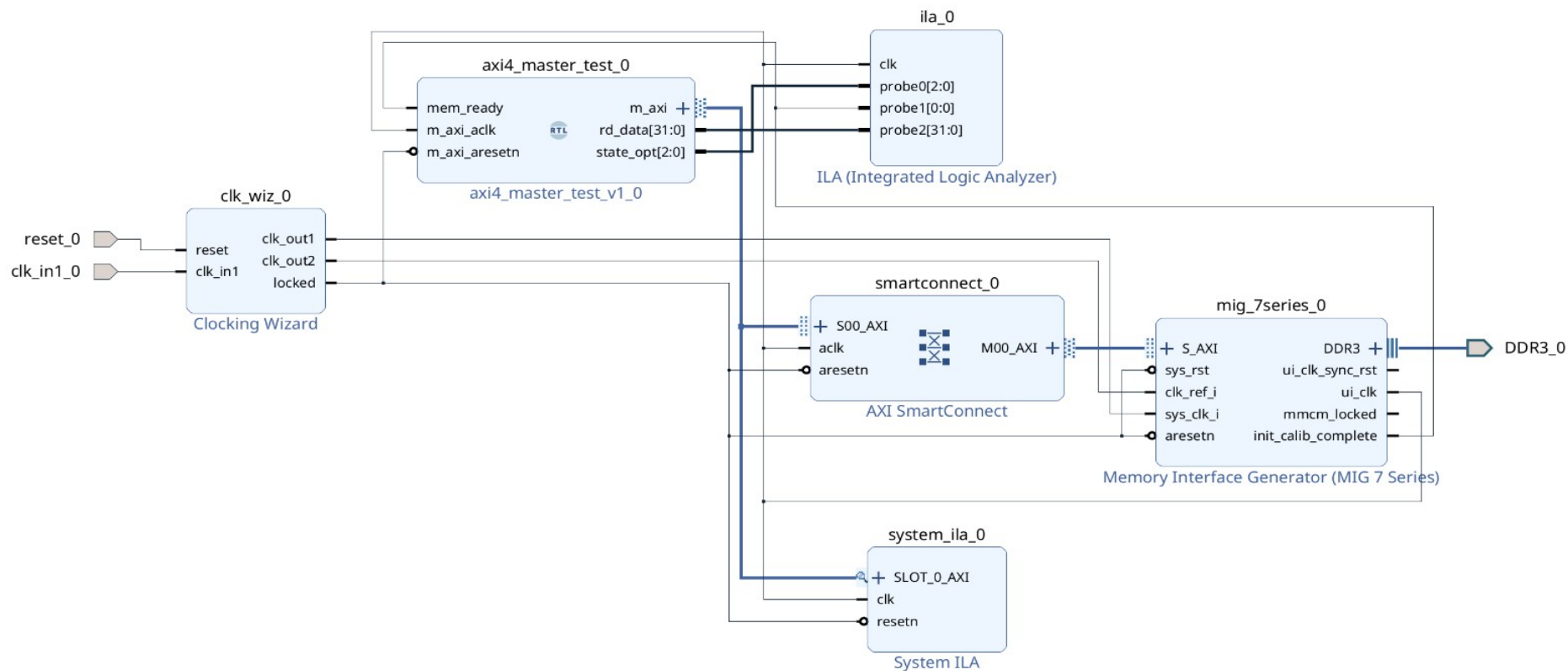
- FIXED:每一拍访问同一地址
- INCR:每一拍访问地址递增(默认)
- WRAP:在到达边界时回绕(边界由AWSIZE的对齐决定,2/4/8/16)



# WSTRB信号

- 位宽与总线的数据位宽相关 (其位宽等于总线数据位宽/8)
- 每一位代表这一字节是否写入
- AXI不允许一拍跨越AWSIZE决定的对齐,但是允许从非对齐地址开始的数据传输
- 此时,第一拍会传输ADDR位于的一个块,需要通过WSTRB控制只写入我们需要的部分
- 同时,数据的位置必须严格与末尾对应

# MIG方案读写DDR



# MIG方案读写DDR

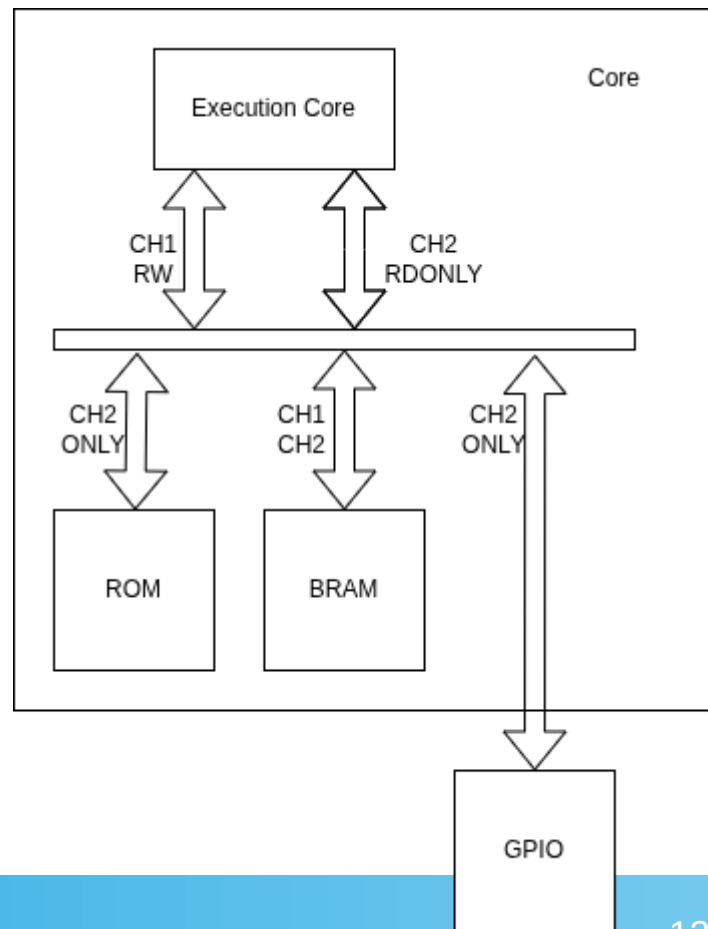
- 配置方法,代码,实践(external)

# 为RISC-V核加入外部AXI控制器

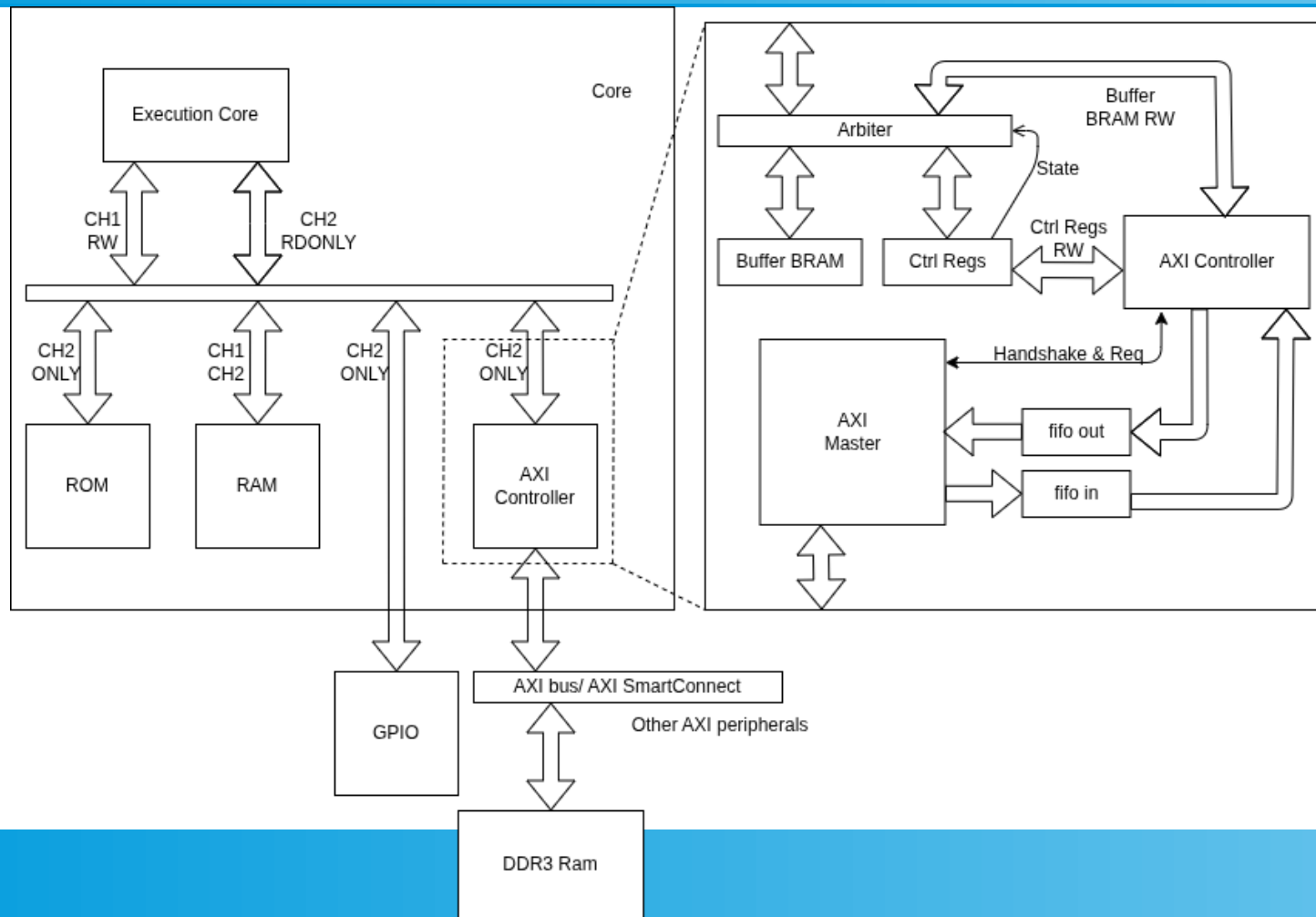
在简化设计CPU时,我希望内部同步读写,符合标准BRAM

也就是:写信号在一周期内完成,读操作在下一周期给出数据

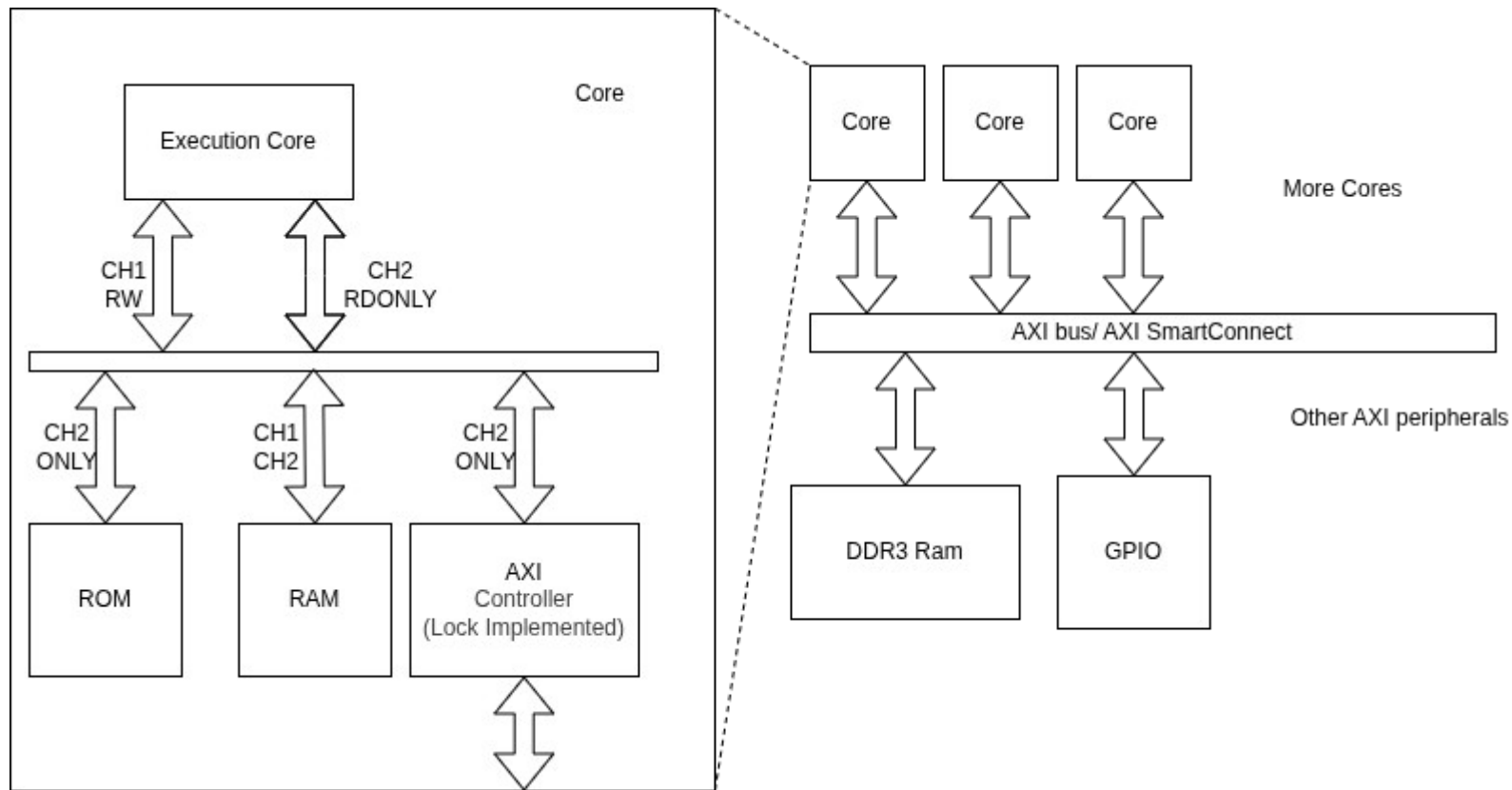
如果引入AXI访问的长延迟,就对我目前而言,貌似就会比较复杂



# 为RISC-V核加入外部AXI控制器



# 多核(未实现)



# 多核(未实现)

还缺少的东西:

- 1. AXI锁
- 2. GPIO AXI slave

已经实现东西中的问题:

- 在不引入AXI模块时,可以跑到100MHZ, 不到200MHz,引入后,目前在83MHz可以跑通